De la programmation Numérique à la Data Intelligence
Rapport de laboratoire

Juin 2024

Xavier Dujardin

Table des matières

Table des matières **2**
Introduction **3**
Contexte 3
Description du projet 3
Approche 3
Exécution du Projet **4**
Phase 1 4
Phase 2 10
Phase 3 13
Résultats 18
Améliorations apportées 18
Conclusions **20**

héna lux
HAUTE ÉCOLE DE NAMUR-LIÈGE-LUXEMBOURG

Introduction

Contexte

Ce projet a été réalisé dans le cadre des laboratoires du cours IE-IG-B2-NPNDI intitulé : “De la programmation numérique à la Data intelligence”, donnés par M. F. Clerin.

La phase 1 ainsi que la phase 2 ont été réalisées par Xavier Dujardin et William Roelens.
La phase 3 a été réalisée de façon individuelle tout en gardant une approche collaborative.
La phase d’amélioration documentée dans ce rapport a été faite de façon individuelle.

Description du projet

Sur base du cours théorique sur l’apprentissage supervisé, nous avons reçu comme objectif de pouvoir reconnaître les mouvements de l’activité humaine via un smartphone. A terme, le programme devra être capable de dire si les données fournies décrivent un mouvement de marche, de course ou encore de montée d’escaliers.

Pour se faire, nous avons reçu un répertoire contenant 15 fois 24 fichiers de données csv contenant les informations enregistrées sur les déplacements de 24 individus. Nous avons aussi reçu un fichier de données csv qui décrit les individus (genre, âge,...).

Le programme a été entièrement créé en C, et des diagrammes d’action ont été fournis.

Approche

L’approche employée lors du début du projet était somme toute assez classique. La recherche de méthodes et de fonctions pour réaliser les phases initiales ayant pris énormément de temps, nous n’avons commencé à réfléchir à la profondeur du projet qu'à partir de la phase 2.

Il nous fallait comprendre ce que la ligne directrice du projet voulait avant de penser plus loin. C’est probablement pour cela que la phase d’améliorations pourra sembler très rigide et plutôt courte.

Exécution du Projet

Phase 1

La première phase du projet concerne la création du test et du train Set. La majeure partie du code est donc dédiée à la récolte d'informations et à l'écriture des sets.

Tout d'abord, des structures de données globales seront définies.
Notées dans le DA de la phase 1, elles seront présentes dans le fichier projet.h

```
InfosLigneSub {
 numLigne,
 attitudeRoll,
 attitudePitch,
 attitudeYaw,
 gravityX,
 gravityY,
 gravityZ,
 rotationRateX,
 rotationRateY,
 rotationRateZ,
 userAccelerationX,
 userAccelerationY,
 userAccelerationZ,
 }

 InfosLigneSet {
 mouvement,
 genre,
 index,
 tabVAcc = ARRAY(600)
 }

 paths = {
 "dws_1/", "dws_2/", "dws_11/", "jog_9/", "jog_16/", "sit_5/",
"sit_13/", "std_6/", "std_14/", "ups_3/", "ups_4/", "ups_12/",
"wlk_7/", "wlk_8/", "wlk_15/"
 }
```

InfosLigneSub est simplement la mise en structure des colonnes de chaque fichiers de données que nous devrons analyser.
InfosLigneSet représente les données des colonnes qui seront dans les Sets

Le tableau paths représente une partie des chemins de chaque fichier sub.

Ensuite, vient les fonctions qui seront utilisées dans la phase

```
o──────────o ↓ path
 │ lireSub │
 o──────────o ↓ infoLigneSet
┌─── *
│ fp = open(path, 'r')
│┌── if (fp == NULL)
│| sortir "Erreur dans l'ouverture du fichier" + path
│|
│├── else
│| row = 0
│|╔══ while ( le fichier a au moins encore une ligne à lire
AND row < 601)
│||  row ++
│||  column = 0
│||┌── if (row == 1)
│|||  continue
│|||
│||├── else
│||| value = séparation des valeurs de la ligne en colonnes
délimitées par ","
│|||╔══ while(value)
│||||
│||||┌── if (column == 0)
│||||| infosLigneSub.numLigne = value
│||||||
│||||├── else if (column == 10)
│||||| infosLigneSub.userAccelerationX = value
│||||||
│||||├── else if (column == 11)
│||||| infosLigneSub.userAccelerationY = value
│||||||
│||||├── else if (column == 12)
│||||| infosLigneSub.userAccelerationZ = value
│||||└──────────
│||||  value = valeur suivante dans la ligne
│||||  column++
│|||└──────────
│|||  infosLigneSet.tabVAcc[row-2] =
racine(infosLigneSub.userAccelerationX**2 +
infosLigneSub.userAccelerationY**2 +
infosLigneSub.userAccelerationY**2);
```

```
    // sécurité si fichier compte moins de 600 lignes
    while(row < 601)
     infosLigneSet.tabVAcc[row-1] = 0;
     row++;
 close(fp)
```

Cette fonction aura pour but de lire un fichier de données et de créer une structure de type InfosLigneSub pour ensuite la transformer en infosLigneSet.
Cette fonction est donc essentielle car elle est la base du projet.

```
 initialisationSet  ↓ path
 *
 fp = open(path, "w")

 écrire dans le fichier : "Mouvement,Genre,Index"
 i = 0
 while (i < 600)
  écrire dans le fichier : ",Vacc"
  i++
 écrire dans le fichier : "\n"
 close(fp)
```

Cette fonction a simplement pour but de créer un set avec une première colonne contenant leur titre. Cette fonction sera utile car elle précède la fonction d'écriture dans un set.

```
o──────────o ↓ path, infosLigneSet
 | ecrireSet |
 o──────────o
┌── *
| fp = open(path, 'a')
|┌── if (fp == NULL)
|| sortir "Erreur dans l'ouverture du fichier" + path
||
|├── else
|| ecrire dans le fichier : infosLigneSet.mouvement + ","
infosLigneSet.genre + "," infosLigneSet.index
||
|| i = 0
||╔══ while (i < 600)
|||┌── if (infosLigneSet.tabVacc[i] == 0)
|||| break;
|||├── else
|||| écrire dans le fichier : "," + infosLigneSet.tabVAcc[i]
|||└──────────
||| i++
||└──────────
|| écrire dans le fichier : "\n"
|└──────────
| close(fp)
└──────────
```

Comme dit au dessus, cette fonction permet d'écrire les informations d'une InfosSet dans le Set.

```
o─────────────────o ↓ tabGenders
 │ getTabGenders │
 o─────────────────o ↓ tabGenders
┌─── *
│ fp = open("data_subjects_info.csv", "r")
│┌── if (fp == NULL)
│| sortir "Erreur dans l'ouverture du fichier" + path
│├── else
│| row = 0
│|╔══ while ( le fichier a au moins encore une ligne à lire)
│||| row++
│||| column = 0
│|||┌── if (row == 1)
│|||| continue
│|||├── else
│|||| value = séparation des valeurs de la ligne en colonnes
délimitées par ","
│||||╔══ while(value)
│|||||┌── if (column == 4)
│|||||| tabGenders[row-2] = value
│|||||└──────────
│|||||| value = valeur suivante dans la ligne
│|||||| column++
│||||└──────────
│|||└──────────
│||└──────────
│└──────────
│ close(fp)
└────────
```

Cette fonction peut être sujet à débat, elle va directement chercher les informations sur les genres des individus dans le fichier data_subjects_info.csv . Nous savons que d'autres groupes auront préféré faire un tableau global mais nous pensons que cette méthode serait plus optimisée si un changement a lieu dans les acteurs.

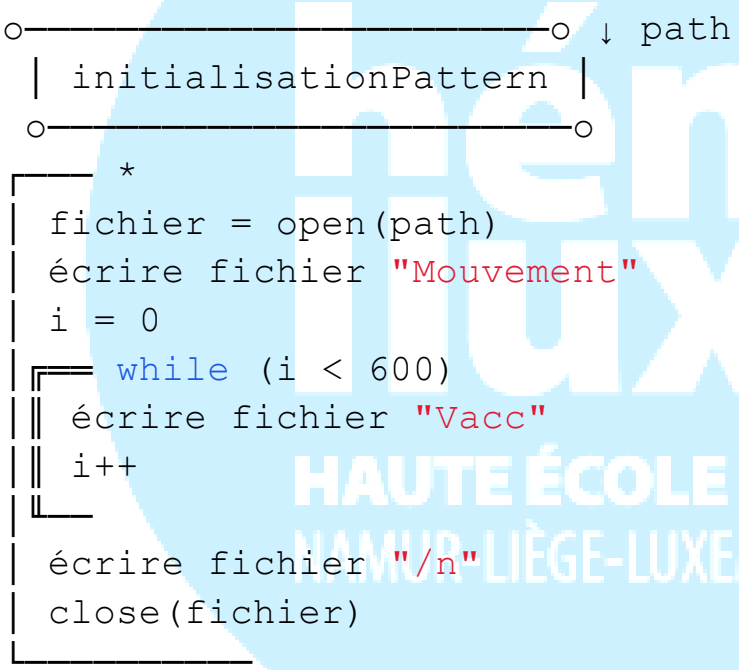
```
o──────o ↓ paths
 │ main │
 o──────o
┌─── *
│ tabGenders = ARRAY(24)
│ getTabGenders(tabGenders)
│ initialisationSet("trainSet.csv")
│ initialisationSet("testSet.csv")
│ path = "A_DeviceMotion_data/A_DeviceMotion_data/"
│ index = 1
│ iDossier = 0
│╔══ while (iDossier < 15)
│╟ iFichier = 1
│╟╔══ while (iFichier < 24)
│╟╟ path += paths[iDossier] + "sub_" + iFichier + ".csv"
│╟╟ infosLigneSet = lireSet(path)
│╟╟ infosLigneSet.genre = tabGenders[iFichier-1]
│╟╟ infosLigneSet.index = index
│╟╟
│╟╟┌── if (iDossier == 0 OR iDossier == 1 OR iDossier == 2)
│╟╟│ infosLigne.mouvement = 1
│╟╟├── else if (iDossier == 3 OR iDossier == 4)
│╟╟│ infosLigne.mouvement = 2
│╟╟├── else if (iDossier == 5 OR iDossier == 6)
│╟╟│ infosLigne.mouvement = 3
│╟╟├── else if (iDossier == 7 OR iDossier == 8)
│╟╟│ infosLigne.mouvement = 4
│╟╟├── else if (iDossier == 9 OR iDossier == 10 OR iDossier == 11)
│╟╟│ infosLigne.mouvement = 5
│╟╟├── else if (iDossier == 12 OR iDossier == 13 OR iDossier ==
14)
│╟╟│ infosLigne.mouvement = 6
│╟╟└──────────
│╟╟
│╟╟┌── if (iFichier % 10 == 0 OR (iFichier == 3  AND
fmod(iDossier,3) == 0) OR (iFichier == 1 AND iDossier == 1))
│╟╟│ ecrireSet("trainSet.csv", infosLigneSet)
│╟╟├── else
│╟╟│ ecrireSet("testSet.csv", infosLigneSet)
│╟╟└──────────
│╟╟ index++
│╟╟ iFichier++
│╟╟ path = "A_DeviceMotion_data/A_DeviceMotion_data/"
│╟└──────────
│╟ iDossier++
│└──────────
└──────────
```

Pour la partie main de cette phase, le code parle de lui-même, nous allons d'abord initialiser les documents et les variables pour ensuite aller continuellement ouvrir un fichier sub pour l'écrire dans le trainSet ou le testSet. On peut noter que c'est dans cette partie que le mouvement pour chaque ligne d'InfosLigne est donné.

Concernant le choix de distribution entre le trainSet et le testSet, nous n'avons pas pu réfléchir à un algorithme. Nous avons alors simplement trouvé une façon d'avoir 10% des fichiers de données dans le trainSet.

Phase 2

La phase 2 se concentre sur la création des patterns, la logique est donc similaire à la phase précédente : récupérer les informations dans un fichier pour la transformer et l'écrire dans un autre fichier.



Idem que dans la phase 1 mais pour le fichier pattern.csv

```
o──────────────o ↓ path, infosLigne
 | lireTrainSet |
 o──────────────o ↓ tabInfosLigneSet
┌── *
| fichier = open(path)
|
|┌── if (fichier == NULL)
|| écrire "Error reading file lireSet"
|├── else
|| row = 0
|| column = 0
|| mouvement = 1
|| tabInfosLigneSet[0].mouvement = 1
|| tabNbMouvements[6] = {0,}
||
||╔══ while (le fichier à encore une ligne à lire)
|||| column = 0
|||| row++
|||┌── if (row == 1)
||||| continue
|||├── else
||||| value = strtok(buffer, ", "
|||||╔══ while (value)
|||||┌── if (column = 0)
|||||| ┌── if (atoi(value) ≠ mouvement)
|||||||| mouvement++
|||||||| tabInfosLigneSet[mouvement-1].mouvement = mouvement
|||||| └──
|||||| tabNbMouvements[mouvement-1]++
|||||├── else if(column = 1)
|||||| infosLigneSet.index = atoi(value)
|||||├── else if(column = 2)
|||||| infosLigneSet.index = atoi(value)
|||||├── else
|||||| info = strtod(value, NULL)
|||||| tabInfosLigneSet[mouvement-1].tabVAcc[column-3] += info
|||| └──
|||| value = strtok(NULL, ", ")
|||| column++
|||╚──
||└──
||
|| mouvement = 0
```

HAUTE ÉCOLE DE NAMUR-LIÈGE-LUXEMBOURG

```
│ │╔══ while (mouvement < 6)
│ │╠ iVacc = 0
│ │╠╗ while (iVacc < 600)
│ │╠╣ tabInfosLigneSet[mouvement].tabVAcc[iVacc] /=
tabNbMouvement[mouvement]
│ │╠╣ iVacc++
│ │╠╝
│ │╠ mouvement++
│ │╚══
│ │
│ │ close(fichier)
│ └──
└──────────
```

Dans la même logique que pour la lecture des sub, on va lire la totalité des informations de chaque ligne du trainSet pour les mettre dans un tableau de structures InfosLigneSet. En oubliant pas de faire la moyenne des vecteurs.

```
o──────────────o ↓ path, infoLigneSet
 │ ecrirePatterns │
 o──────────────o
┌─── *
│ fichier = open(path)
│
│┌── if (fichier == NULL)
│└ écrire "Error reading file ecriturePatterns"
│┌─ else
│└ // impression pour chaque ligne
│ écrire fichier "" + infosLigneSet.mouvement
│ i = 0
│╔══ while (i < 600)
│╠╗┌── if (infosLigneSet.tabVAcc[i] == 0)
│╠╣└ break
│╠╣┌─ else
│╠╣└ écrire fichier "" + infosLigneSet.tabVAcc[i]
│╠╝└──
│╠ i++
│╚══
│ écriture fichier "/n"
│ close(fichier)
└──
└──────────
```

Ici, on va écrire dans le pattern.csv toutes les informations contenues dans une structure InfosLigneSet

```
Main
  *
  initialisationPattern ↓ 'pattern.csv'
tabTrainSet = ARRAY(6)
iMouvement = 0
while (i < 6)
  iVacc = 0
  while(iVacc < 600)
    tabTrainSet[iMouvement].tabVacc[iVacc] = 0
    iVacc++
  iMouvement++
  lireTrainSet ↓ 'trainSet.csv', tabTrainSet
  ↓ tabTrainSet
iLigne = 0
while(iLigne < 6)
  ecrirePattens ↓ 'pattern.csv', tabTrainSet[iLigne]
  iLigne++
```

Pour cette fonction principale, le but est simple, initialiser le pattern.csv et le tabTrainSet. Ensuite, on remplit le tabTrainSet pour finalement garnir le pattern.csv.

Phase 3

L'objectif de la 3ème phase est de déterminer si les valeurs des patterns correspondent avec les valeurs du testSet pour finalement donner un taux d'efficacité du programme.

Comme il faut récupérer les données du pattern.csv, une nouvelle structure apparait :

```
InfoLignePattern {
 mouvement,
 vAcc = ARRAY(600)
 }
```



Encore une fois, la méthode de lecture du fichier ne change pas, les informations de celui-ci sont notées dans le tableau de structure de type InfoLignePattern *patterns.*



Pareil pour le testSet, les données iront dans le tabInfosLigneSet pour pouvoir etre ensuite comparées aux données du tableau patterns

```
o──────────────────────────────o ↓ realClasses,
estimatedClasses, nbTests
 │ displayResultForEachClass │
 o──────────────────────────────o
┌─── *
│ sortir 'REAL CLASSES\tESTIMATED CLASSES'
│ i = 0
│╔══ while (i < nbTests)
│╟ sortir '\n' + (i+1) + ' : ' + realClasses[i] + '\t\t' +
estimatedClassed[i]
│╟ i++
│└──────────
└──────────

 o──────────────────o ↓ realClasses, estimatedClasses,
nbTests
 │ displayAccuracy │
 o──────────────────o
┌─── *
│ accuracy = 0
│ i = 0
│╔══ while( i < nbTests)
│╟┌── if (realClasses[i] == estimatedClasses[i])
│╟│ accuracy++
│╟└──────────
│╟ nbTests++
│└──────────
└──────────
```

Ces deux fonctions servent à l'affichage des résultats, la première va donner la classe réelle de la donnée avec la classe estimée par le programme.

La seconde va afficher le taux de réussite. Une réussite est définie par l'égalité entre la classe réelle et la classe estimée.

```
o──────o
 | Main |
 o──────o
┌─── *
│ fiTest = open('testSet.csv', 'r')
│ fiPattern = open('pattern.csv', 'r')
│┌── if (fiTest AND fiPattern)
│| models = ARRAY(6)
│| o──────────────o ↓ fiPattern, models
│| | lirePattern |
│| o──────────────o ↓ models
│| testSet = ARRAY(36)
│| o──────────────o ↓ fiTest, testSet
│| | lireTestSet |
│| o──────────────o ↓ testSet
│|
│| realClasses = ARRAY(36)
│| i = 0
│|╔═ while(i < 36)
│||  realClasses[i] = testSet[i].mouvement
│||  i++
│|└──────────
│|
│| estimatedClasses[36] = {0,}
│| iLigne = 0
│|╔═ while(iLigne < 36)
│||  dist = HV
│||  iModel = 0
│||╔═ while(iModel < 6)
│|||  temp = 0
│|||  iVacc = 0
│|||╔═ while(iVacc < 600)
│||||  temp += (testSet[iLigne].tabVAcc[iVacc] -
models[iModel].vAcc[iVacc])**2
│||||  iVacc++
│|||└──────────
│|||  temp = sqrt(temp)
│|||┌── if (temp ≤ dist)
│||||  dist = temp
│||||  estimatedClasses[iLigne] = iModel + 1
│|||└──────────
│|||  iModel++
│||└──────────
```

```
    iLigne++
    displayResultForEachClass(realClasses,estimatedClasses,36)
    displayAccuracy(realClasses,estimatedClasses,36)
else
    sortir 'erreur dans l'ouverture de fiTest ou fiPattern'
```

Pour la fonction principale, on y aura remarqué l'algorithme de comparaison entre les données des patterns et celles du testSet sur base de la distance euclidienne.

Résultats

```
REAL CLASSES    ESTIMATED CLASSES
1 : 1           5
2 : 1           5
3 : 1           5
4 : 1           5
5 : 1           1
6 : 1           5
7 : 1           1
8 : 1           1
9 : 2           2
10 : 2          2
11 : 2          2
12 : 2          2
13 : 2          2
14 : 3          3
15 : 3          3
16 : 3          4
17 : 3          4
18 : 3          4
19 : 4          4
20 : 4          4
21 : 4          4
22 : 4          4
23 : 5          5
24 : 5          5
25 : 5          5
26 : 5          5
27 : 5          5
28 : 5          5
29 : 5          5
30 : 6          5
31 : 6          5
32 : 6          5
33 : 6          5
34 : 6          6
35 : 6          1
36 : 6          1
Accuracy : 61.111111 22
```

Avec le programme réalisé, nous avons atteint les 61,11% de réussite (accuracy)

Le problème étant que la distribution des données entre le testSet et le trainSet est très peu recherchée. Nous en avons donc déduit que cela relevait de la "chance" car les fichiers choisis étaient miraculeusement les bons.
Néanmoins, cela nous donne une piste de réflexion pour d' éventuelles améliorations futures.

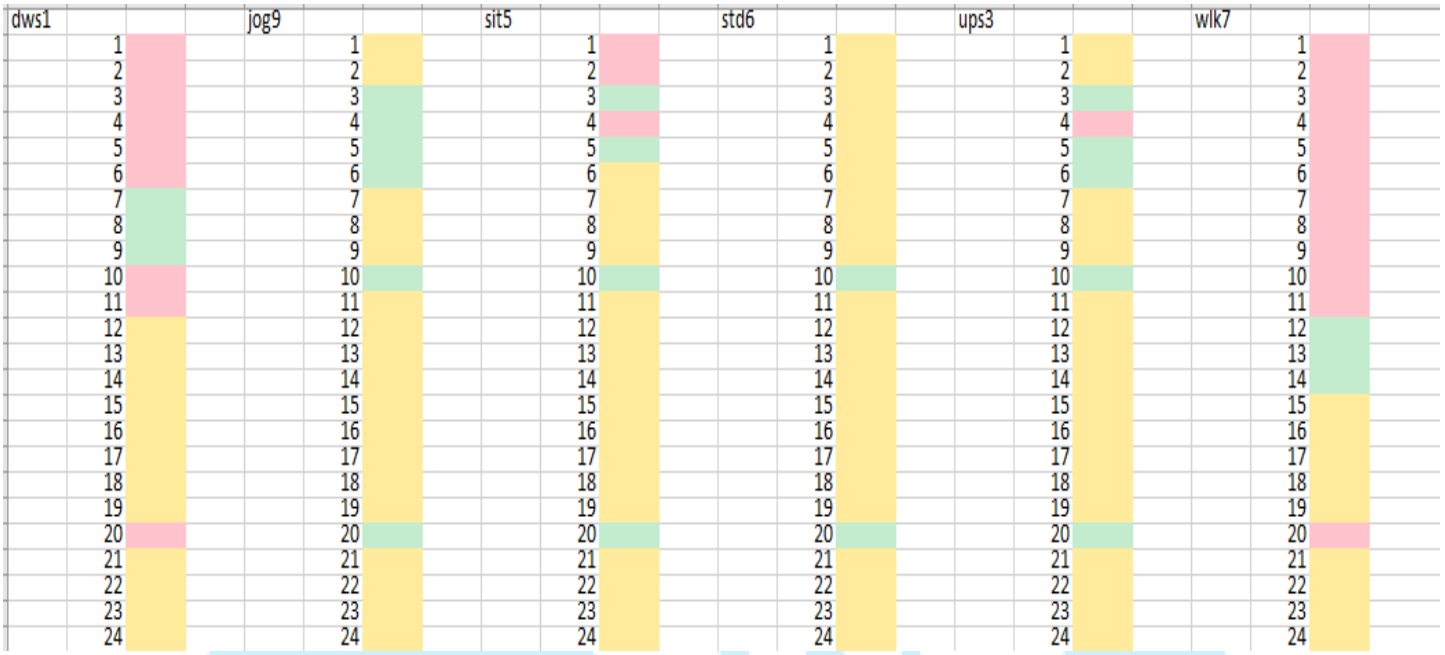
Améliorations apportées

Etant donné que nous avons directement songé à la chance quant à notre taux de réussite dans la phase 3, j'ai décidé de tester fichier par fichier.

La méthode de distribution à ce moment là étant :

```
if(
iFichier % 10 == 0 ||
(iFichier == 3  && fmod(iDossier,3)
== 0) ||
(iFichier == 1 && iDossier == 1)
)
```

J’ai alors décidé de changer le dernier paramètre de cette proposition en testant donc tous les fichiers de chaque dossier un par un. En répertoriant les résultats dans un tableau excel, je suis rapidement arrivé à un constat



dws1	jog9	sit5	std6	ups3	wlk7
1	1	1	1	1	1
2	2	2	2	2	2
3	3	3	3	3	3
4	4	4	4	4	4
5	5	5	5	5	5
6	6	6	6	6	6
7	7	7	7	7	7
8	8	8	8	8	8
9	9	9	9	9	9
10	10	10	10	10	10
11	11	11	11	11	11
12	12	12	12	12	12
13	13	13	13	13	13
14	14	14	14	14	14
15	15	15	15	15	15
16	16	16	16	16	16
17	17	17	17	17	17
18	18	18	18	18	18
19	19	19	19	19	19
20	20	20	20	20	20
21	21	21	21	21	21
22	22	22	22	22	22
23	23	23	23	23	23
24	24	24	24	24	24

Certains déplacements sont plus faciles à déterminer que d’autres.
Il est évident que continuer des tests sur la course est inutile, les résultats étant tellement éloignés des autres qu' il n'y a aucun risque d’erreur.

Avec les résultats affichés précédemment, on remarque que la descente d’escaliers(1) est souvent confondue avec la montée d’escaliers (5). La marche peut se confondre avec la montée ou la descente d’escaliers. Pour finir, il arrive que la position assise (3) se confonde avec la position debout (4)

Le but est donc maintenant de trouver des données qui ne s'écartent pas trop de leur pattern car il peut exister des aberrations.

Malheureusement, je n’ai pas eu suffisamment de temps pour mettre en application mon idée mais je peux aisément supposer que la partie 1 complémentaire avait une utilité bien plus grande qu’on ne le pensait.
Il aurait fallu recenser les fichiers dont les données étaient trop écartées de leur moyenne respective.

Il aurait ensuite fallu adapter la sélection des fichiers en fonction de ce rapport, en ne prenant donc pas en compte les fichiers notés comme “aberrant”.

On peut également envisager de tester chaque individu afin de détecter ceux qui seraient le plus susceptibles de créer des aberrations dans les données. En effet, chaque être humain a une façon unique de se déplacer et certains pourraient entrer en conflit avec les données d’autres sujets pour d’autres déplacements. Je n’ai pas vraiment d’idée pour concrétiser cette pensée sur un plan algorithmique.

Enfin, une solution qui me semble claire est l'utilisation des autres données présentes dans les fichiers sub.
Reprenons la structure InfosLigneSub définie dans la phase 1

```
InfosLigneSub {
numLigne,
attitudeRoll,
attitudePitch,
attitudeYaw,
gravityX,
gravityY,
gravityZ,
rotationRateX,
rotationRateY,
rotationRateZ,
userAccelerationX,
userAccelerationY,
userAccelerationZ,
 }
```

On remarque facilement de multiples vecteurs. Mais seuls les vecteurs d'accélérations (rouge) seront utilisés. On peut donc facilement conclure que les résultats auraient été bien plus précis si les autres vecteurs avaient été pris en compte dans le processus d'évaluation. Maintenant je comprends que le cours ne nous ait pas demandé d'en faire usage au vu de la complexité mathématique qui aurait, selon moi, dépassé l'objet du cours.

HAUTE ÉCOLE DE NAMUR-LIÈGE-LUXEMBOURG

Conclusions

En suivant le principe de test de chaque fichier, on arrive à un taux de réussite de 94% avec cette disposition de fichiers sélectionnés :

```
if((iDossier == 0 && (iFichier == 7 || iFichier == 8 || iFichier == 9))
        || (iDossier == 1 && (iFichier == 8 || iFichier == 10))
        || (iDossier == 2 && (iFichier == 9 || iFichier == 10 || iFichier == 20))
        || (iDossier == 3 && (iFichier == 3 || iFichier == 4 || iFichier == 5))
        || (iDossier == 4 && (iFichier == 1 || iFichier == 10 || iFichier == 20))
        || (iDossier == 5 && (iFichier == 3 || iFichier == 5 || iFichier == 10))
        || (iDossier == 6 && (iFichier == 5 || iFichier == 7))
        || (iDossier == 7 && (iFichier == 10 || iFichier == 20))
        || (iDossier == 8 && (iFichier == 10 || iFichier == 20))
        || (iDossier == 9 && (iFichier == 3 || iFichier == 5))
        || (iDossier == 10 && (iFichier == 10 || iFichier == 20))
        || (iDossier == 11 && (iFichier == 10 || iFichier == 20))
        || (iDossier == 12 && (iFichier == 12 || iFichier == 13 || iFichier == 14))
```

```
        || (iDossier == 13 && (iFichier == 21 || iFichier == 1))
        || (iDossier == 14 && (iFichier == 1 || iFichier == 2)))
```

```
REAL CLASSES    ESTIMATE
1 : 1           1
2 : 1           1
3 : 1           1
4 : 1           1
5 : 1           1
6 : 1           1
7 : 1           1
8 : 1           1
9 : 2           2
10 : 2          2
11 : 2          2
12 : 2          2
13 : 2          2
14 : 2          2
15 : 3          3
16 : 3          3
17 : 3          3
18 : 3          3
19 : 3          3
20 : 4          4
21 : 4          4
22 : 4          4
23 : 4          4
24 : 5          5
25 : 5          5
26 : 5          5
27 : 5          5
28 : 5          5
29 : 5          5
30 : 6          6
31 : 6          6
32 : 6          6
33 : 6          6
34 : 6          5
35 : 6          1
36 : 6          6
Accuracy : 94.444444 34
```

Je pense que ces résultats sont assez satisfaisants, cependant je trouve que le moyen d'y arriver est décevant, il nous aurait fallu plus de temps pour arriver à une meilleure fin.

En définitive, le programme arrive à reconnaître la consistance des données initialement fournies et cela est plutôt intriguant. La complexité du projet rend ces résultats d'autant plus intéressants. Nous avons réussi à, dans un certain sens, faire "parler" des données via des algorithmes. De plus, la phase de réflexion imposée par la création de ce rapport montre les multiples possibilités d'amélioration du projet et la complexité profonde de celui-ci.

héna
lux
HAUTE ÉCOLE DE
HAINAUT-NAMUR-LIÈGE-LUXEMBOURG